Recursividade e Análise de Recorrências

Resumo completo — baseado em `Aula5_AnaliseRecursividade.pdf` e `Aula5_AnaliseRecursividade_GABARITO.pdf` (Análise e Complexidade de Algoritmos).

Objetivo do tema: olhar para um algoritmo recursivo, transformar o código em uma **equação de recorrência** e resolver essa equação para obter a complexidade.

Conteúdo

0. Revisão: notação assintótica
1. O que é recursividade
2. Contagem de passos elementares
3. Como construir uma recorrência
4. Método da iteração (expansão)
5. Método da árvore de recursão
6. Método da substituição (prova)
7. Método Mestre
8. Recursão com subproblemas diferentes: Fibonacci
9. Complexidade de espaço: a pilha de recursão
10. Qual método usar?
11. Exercícios resolvidos (Praticando 1 a 11)
12. Matérias correlatas
13. Apêndice: ferramentas matemáticas

---

0. Revisão: notação assintótica

Não interessa o custo exato, e sim **como o custo cresce** quando n aumenta.

Notação	Significado	Definição (existem constantes $c > 0$ e n_0 tais que, para todo $n \geq n_0$...)
$T(n) = O(g(n))$	limite **superior** ("no máximo")	$T(n) \leq c \cdot g(n)$
$T(n) = \Omega(g(n))$	limite **inferior** ("no mínimo")	$T(n) \geq c \cdot g(n)$
$T(n) = \Theta(g(n))$	limite **justo** ("exatamente a ordem")	vale $O(g(n))$ **e** $\Omega(g(n))$

Por isso, na aula, "$O + \Omega = \Theta$": para concluir Θ é preciso provar os dois lados.

Ordem de crescimento (do mais eficiente ao menos eficiente):

$$\Theta(1) < \Theta(\log n) < \Theta(n) < \Theta(n \log n) < \Theta(n^2) < \Theta(2^n)$$

---

1. O que é recursividade

Uma **função recursiva** chama a si mesma para resolver instâncias menores do mesmo problema. Ela precisa de:

- **Caso base:** condição que interrompe a recursão (sem ele, a recursão nunca termina).
- **Caso recursivo:** a função chama a si mesma com um problema **menor**.
- **Progresso:** cada chamada deve caminhar em direção ao caso base.

Exemplo — fatorial:

```
def fatorial(n):
    if n == 0:            # caso base
        return 1
    return n * fatorial(n - 1)   # caso recursivo
```

Execução para $n = 4$:

```
fatorial(4)
  = 4 * fatorial(3)
      = 3 * fatorial(2)
          = 2 * fatorial(1)
              = 1 * fatorial(0)
                  = 1           (caso base)
```

Foram **5 chamadas** para $n = 4$. Em geral, $n + 1$ chamadas, cada uma com trabalho constante, logo $\Theta(n)$.

---

2. Contagem de passos elementares

Um **passo elementar** é uma operação de custo constante c: atribuição, comparação, operação aritmética, acesso a uma posição, chamada de função.

Algoritmo iterativo: basta contar as repetições do laço.

```
for i in range(n):
    print(i)        # executa n vezes
```

$T(n) = c \cdot n$, ou seja, $T(n) = \Theta(n)$.

Algoritmo recursivo: uma chamada gera outras chamadas, que geram outras... Não dá para "contar o laço" diretamente. A ferramenta para representar isso é a **equação de recorrência**.

```
def soma(n):
    if n == 1:
        return 1
    return n + soma(n - 1)
```

Cada chamada faz uma comparação, uma soma e uma chamada recursiva: custo constante c. Então:

$$T(n) = T(n-1) + c, \qquad T(1) = c_0$$

Leitura: *para resolver um problema de tamanho n, resolvo um de tamanho $n-1$ e gasto mais um custo constante c.*

---

3. Como construir uma recorrência

Forma geral:

$$T(n) = \underbrace{\text{custo das chamadas recursivas}}_{a \cdot T(\text{tamanho do subproblema})} + \underbrace{\text{trabalho local}}_{f(n)}$$

Receita — responda 4 perguntas:

1. **Quantas chamadas recursivas existem?** → coeficiente a
2. **Qual o tamanho de cada subproblema?** → $n-1$, $n/2$, $n/4$...
3. **Quanto trabalho é feito fora da recursão?** → $f(n)$ (laços, comparações, intercalação...)
4. **Qual é o caso base?** → $T(1) = c_0$ (constante)

Tabela de tradução código → recorrência:

Padrão no código	Recorrência
1 chamada `f(n-1)` + trabalho constante	$T(n) = T(n-1) + c$
1 chamada `f(n-1)` + laço de n	$T(n) = T(n-1) + n$
1 chamada `f(n//2)` + trabalho constante	$T(n) = T(n/2) + c$
1 chamada `f(n//2)` + laço de n	$T(n) = T(n/2) + n$
2 chamadas `f(n//2)` + trabalho constante	$T(n) = 2T(n/2) + c$
2 chamadas `f(n//2)` + laço de n	$T(n) = 2T(n/2) + n$
`f(n//2)` e `f(n//4)` + laço de n	$T(n) = T(n/2) + T(n/4) + n$
`f(n-1)` e `f(n-2)` + constante	$T(n) = T(n-1) + T(n-2) + c$

> **Atenção:** o número de chamadas, sozinho, **não** determina a complexidade. É preciso considerar também o tamanho dos subproblemas e o trabalho local (veja Praticando 5 e 10).

---

4. Método da iteração (expansão)

Ideia: substituir a recorrência dentro dela mesma várias vezes até enxergar um padrão.

Os 6 passos:

1. Escrever a recorrência;
2. Expandir a recorrência;
3. Identificar um padrão;
4. Generalizar (escrever o termo após k expansões);
5. Encontrar a condição de parada (quando chega ao caso base);
6. Determinar a complexidade.

4.1 Exemplo 1 — recursão linear: $T(n) = T(n-1) + c$

```
def exemplo(n):
    if n <= 1:
        return 1
    return exemplo(n - 1) + 1
```

Uma chamada `exemplo(n-1)` e trabalho constante: $T(n) = T(n-1) + c$, $T(1) = c_0$.

Passo 2 — expandir. Como $T(n-1) = T(n-2) + c$:

$$T(n) = [T(n-2) + c] + c = T(n-2) + 2c$$

Como $T(n-2) = T(n-3) + c$:

$$T(n) = [T(n-3) + c] + 2c = T(n-3) + 3c$$

Passos 3 e 4 — padrão após k expansões:

$$T(n) = T(n-k) + k\,c$$

Passo 5 — parada: a recursão termina quando $n - k = 1$, isto é, $k = n - 1$.

Passo 6 — substituir:

$$T(n) = T(1) + (n-1)\,c = c_0 + (n-1)\,c$$

É uma função linear de n: $\boxed{T(n) = \Theta(n)}$.

4.2 Exemplo 2 — custo crescente: $T(n) = T(n-1) + n$

```
def soma_recursiva(n):
    if n <= 1:
        return 1
    for i in range(n):      # trabalho local: n passos
        print(i)
    return soma_recursiva(n - 1)   # uma chamada recursiva
```

$T(n) = T(n-1) + n$, com $T(1) = c$.

Expandindo: $T(n-1) = T(n-2) + (n-1)$, logo

$$T(n) = T(n-2) + (n-1) + n$$

$T(n-2) = T(n-3) + (n-2)$, logo

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

Após k expansões:

$$T(n) = T(n-k) + (n-k+1) + \cdots + (n-1) + n$$

Parada: $n - k = 1 \Rightarrow k = n - 1$:

$$T(n) = T(1) + 2 + 3 + \cdots + n$$

Usando a soma de Gauss (apêndice), $1 + 2 + \cdots + n = \dfrac{n(n+1)}{2}$, então $2 + 3 + \cdots + n = \dfrac{n(n+1)}{2} - 1$:

$$T(n) = c + \frac{n(n+1)}{2} - 1 = \frac{n^2}{2} + \frac{n}{2} + (c - 1)$$

O termo dominante é $n^2/2$: $\boxed{T(n) = \Theta(n^2)}$.

4.3 Divisão pela metade com custo constante: $T(n) = T(n/2) + c$

(É o padrão da **busca binária** e do Praticando 1B / 9.)

$$\begin{aligned} T(n) &= T(n/2) + c \\ &= T(n/4) + 2c \\ &= T(n/8) + 3c \\ &\vdots \\ &= T(n/2^k) + k\,c \end{aligned}$$

Parada: $n/2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$.

$$T(n) = T(1) + c\log_2 n \quad \Rightarrow \quad \boxed{T(n) = \Theta(\log n)}$$

4.4 Divisão pela metade com custo linear: $T(n) = T(n/2) + n$

(Praticando 2.)

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + \cdots + \frac{n}{2^{k-1}} + T\left(\frac{n}{2^k}\right)$$

Com $k = \log_2 n$, a soma é uma **série geométrica** de razão $1/2$ (apêndice):

$$n\sum_{i=0}^{k-1}\left(\frac{1}{2}\right)^i < n\sum_{i=0}^{\infty}\left(\frac{1}{2}\right)^i = n \cdot \frac{1}{1-1/2} = 2n$$

Então $T(n) \leq 2n + T(1) = O(n)$. E como só o primeiro nível já custa n, $T(n) = \Omega(n)$. Portanto $\boxed{T(n) = \Theta(n)}$.

> **Lição:** haver recursão **não** significa $n \log n$. Aqui os custos encolhem pela metade a cada nível, e a soma é dominada pelo primeiro termo.

---

5. Método da árvore de recursão

Ideia: desenhar cada chamada como um nó, anotar o custo local de cada nó e **somar por nível**.

Roteiro:

1. Quantos nós há no nível i?
2. Qual o tamanho do subproblema de cada nó no nível i?
3. Custo do nível i = (nº de nós) × (custo local de cada nó).
4. Quantos níveis existem? (quando o tamanho chega a 1)
5. Somar os custos de todos os níveis.

5.1 Exemplo 3: $T(n) = 2T(n/2) + n$

```
def processo(n):
    if n <= 1:
        return
    processo(n // 2)        # 2 chamadas recursivas,
    processo(n // 2)        # cada uma de tamanho n/2
    for i in range(n):      # trabalho local: n
        print(i)
```

```
Nível 0:                    n                        1 nó  × n      = n
                          /     \
Nível 1:              n/2         n/2                2 nós × n/2    = n
                    /   \       /   \
Nível 2:          n/4   n/4   n/4   n/4              4 nós × n/4    = n
                  ...                              ...
Nível k:      n/2^k  ...  (2^k nós)                  2^k × n/2^k    = n
```

- **Custo de cada nível:** $2^i \cdot \dfrac{n}{2^i} = n$ (todo nível custa n).
- **Número de níveis:** o tamanho chega a 1 quando $\dfrac{n}{2^k} = 1 \Rightarrow k = \log_2 n$. Há $\log_2 n$ níveis de divisão (ou $\log_2 n + 1$ contando o nível das folhas).
- **Soma:**

$$T(n) = \underbrace{n + n + \cdots + n}_{\log_2 n \text{ níveis}} + \underbrace{n \cdot T(1)}_{\text{folhas}} = n \log_2 n + c_0\, n$$

$$\boxed{T(n) = \Theta(n \log n)}$$

> A árvore permite **formular uma hipótese**. Visualizar não é provar: a prova formal vem pelo **Método da Substituição** (seção 6) ou pelo **Método Mestre** (seção 7).

5.2 $T(n) = 2T(n/2) + 1$ (Praticando 4)

Nível i	Nº de nós	Custo de cada nó	Custo do nível
0	1	1	1
1	2	1	2
2	4	1	4
3	8	1	8
i	2^i	1	2^i

Níveis: $i = 0, 1, \ldots, \log_2 n$. Soma geométrica de razão 2:

$$T(n) = \sum_{i=0}^{\log_2 n} 2^i = \frac{2^{\log_2 n + 1} - 1}{2 - 1} = 2n - 1$$

$$\boxed{T(n) = \Theta(n)}$$

Aqui o custo **cresce** a cada nível; quem domina são as **folhas** (n folhas).

5.3 Subproblemas de tamanhos diferentes: $T(n) = T(n/2) + T(n/4) + n$ (Praticando 3)

```
Nível 0:                n                                  custo n
                      /     \
Nível 1:          n/2         n/4                          n/2 + n/4 = (3/4) n
                 /   \       /   \
Nível 2:       n/4   n/8   n/8   n/16                       (9/16) n = (3/4)^2 n
```

Nível	Tamanhos	Custo do nível
0	n	cn
1	$n/2,\ n/4$	$\frac{3}{4}cn$
2	$n/4,\ n/8,\ n/8,\ n/16$	$\frac{9}{16}cn$
3	$n/8, n/16, n/16, n/32, n/16, n/32, n/32, n/64$	$\frac{27}{64}cn$
k	...	$\left(\frac{3}{4}\right)^k cn$

Cada nível custa $3/4$ do anterior. Somando (série geométrica com $0 < 3/4 < 1$):

$$T(n) \leq cn \sum_{k=0}^{\infty} \left(\frac{3}{4}\right)^k = cn \cdot \frac{1}{1 - 3/4} = 4cn$$

Logo $T(n) = O(n)$; como o nível 0 já custa cn, $T(n) = \Omega(n)$. Portanto $\boxed{T(n) = \Theta(n)}$.

> O Método Mestre **não** se aplica diretamente aqui, porque os subproblemas têm tamanhos diferentes.

5.4 Quando as folhas dominam muito: $T(n) = 4T(n/2) + n$

Nível i: 4^i nós, cada um de tamanho $n/2^i$ → custo $4^i \cdot \dfrac{n}{2^i} = 2^i\, n$.

$$T(n) = \sum_{i=0}^{\log_2 n} 2^i n = n\left(2^{\log_2 n + 1} - 1\right) = n(2n - 1) = 2n^2 - n = \Theta(n^2)$$

Resultado confirmado pelo Método Mestre, Caso 1 (seção 7).

Resumo do comportamento da árvore

Custos por nível	Quem domina	Resultado
Diminuem geometricamente	a raiz	$\Theta(f(n))$
Iguais em todos os níveis	todos os níveis	$\Theta(f(n) \cdot \log n)$
Aumentam geometricamente	as folhas	$\Theta(\text{nº de folhas}) = \Theta(n^{\log_b a})$

Essa tabela é exatamente a intuição dos **três casos do Método Mestre**.

---

6. Método da substituição (prova)

Duas etapas:

1. **Formular uma hipótese** (a árvore ou a iteração sugerem o resultado);
2. **Provar a hipótese** por indução, substituindo-a na própria recorrência.

6.1 Provando $T(n) = 2T(n/2) + n = O(n \log n)$

Hipótese: $T(m) \leq c\, m \log m$ para todo $m < n$ (logaritmo na base 2).

Substituindo $T(n/2)$ pela hipótese:

$$T(n/2) \leq c\,\frac{n}{2}\log\frac{n}{2}$$

$$T(n) \leq 2\left(c\,\frac{n}{2}\log\frac{n}{2}\right) + n = c\,n\log\frac{n}{2} + n$$

Como $\log\dfrac{n}{2} = \log n - \log 2 = \log n - 1$:

$$T(n) \leq cn(\log n - 1) + n = cn\log n - cn + n$$

Para $c \geq 1$: $-cn + n \leq 0$, logo

$$T(n) \leq cn\log n \quad \Rightarrow \quad T(n) = O(n\log n)$$

Detalhe técnico (caso base): para $n = 1$, $c \cdot 1 \cdot \log 1 = 0$, que não limita $T(1) = c_0 > 0$. Resolve-se começando a indução em $n = 2$ e $n = 3$ (escolhendo c grande o bastante, por exemplo $c \geq c_0 + 1$), o que é permitido, pois a notação O só exige valer para $n \geq n_0$.

6.2 Provando o limite inferior $\Omega(n \log n)$

Para uma prova formal completa de Θ é necessário demonstrar também Ω — é matematicamente mais correto do que concluir Θ só a partir da prova de O.

Hipótese: $T(m) \geq d\, m \log m$ para $m < n$.

$$T(n) \geq 2\left(d\,\frac{n}{2}\log\frac{n}{2}\right) + n = dn\log n - dn + n$$

Para $0 < d \leq 1$: $-dn + n \geq 0$, logo $T(n) \geq dn\log n$, isto é, $T(n) = \Omega(n\log n)$.

Conclusão: $O(n\log n)$ e $\Omega(n\log n) \Rightarrow \boxed{T(n) = \Theta(n\log n)}$.

6.3 Armadilha clássica: "provar" um palpite errado

Suponha o palpite (errado) $T(n) = 2T(n/2) + n = O(n)$, ou seja, $T(m) \leq cm$:

$$T(n) \leq 2\left(c\,\frac{n}{2}\right) + n = cn + n$$

$cn + n$ **não** é $\leq cn$ para nenhum c (sobra o $+n$). A indução **falha**: é preciso provar a forma **exata** da hipótese, não "algo parecido". Essa é a resposta formal ao aluno do Praticando 10.

---

7. Método Mestre

Aplica-se a recorrências da forma

$$T(n) = a\,T(n/b) + f(n), \qquad a \geq 1,\ b > 1$$

- a: número de subproblemas;
- n/b: tamanho de cada subproblema (**todos do mesmo tamanho**);
- $f(n)$: trabalho fora da recursão.

Ideia: comparar $f(n)$ com $n^{\log_b a}$ (que é o número de folhas da árvore).

Caso	Comparação (forma simplificada)	Condição formal	Resultado
1	$f(n) < n^{\log_b a}$	$f(n) = O(n^{\log_b a - \varepsilon})$ para algum $\varepsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = n^{\log_b a}$	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$
3	$f(n) > n^{\log_b a}$	$f(n) = \Omega(n^{\log_b a + \varepsilon})$ **e** regularidade: $a\,f(n/b) \leq k\,f(n)$ com $k < 1$	$T(n) = \Theta(f(n))$

> **Condições técnicas:** nos casos 1 e 3, "menor" e "maior" significam **polinomialmente** menor/maior (por um fator n^{ε}). O caso 3 exige ainda a **condição de regularidade**. Exemplo fora dos três casos: $T(n) = 2T(n/2) + n\log n$ ($n\log n$ é maior que n, mas não polinomialmente).

Receita passo a passo

1. Identifique a, b e $f(n)$.
2. Calcule $n^{\log_b a}$.
3. Compare com $f(n)$ e escolha o caso.
4. (Caso 3) Verifique a regularidade.
5. Escreva o resultado.

Exemplos resolvidos

Caso 1 — $T(n) = 4T(n/2) + n$

1. $a = 4$, $b = 2$, $f(n) = n$
2. $n^{\log_2 4} = n^2$
3. $n = O(n^{2-\varepsilon})$ com $\varepsilon = 1$ → Caso 1
4. $\boxed{T(n) = \Theta(n^2)}$

Caso 2 — $T(n) = 2T(n/2) + n$ (Exemplo 3, Merge Sort)

1. $a = 2$, $b = 2$, $f(n) = n$
2. $n^{\log_2 2} = n$
3. $f(n) = \Theta(n)$ → Caso 2
4. $\boxed{T(n) = \Theta(n \log n)}$

Caso 3 — $T(n) = 2T(n/2) + n^2$

1. $a = 2$, $b = 2$, $f(n) = n^2$
2. $n^{\log_2 2} = n$
3. $n^2 = \Omega(n^{1+\varepsilon})$ com $\varepsilon = 1$ → Caso 3
4. Regularidade: $af(n/b) = 2\left(\frac{n}{2}\right)^2 = \frac{n^2}{2} \leq \frac{1}{2}n^2$ ✓ ($k = 1/2$)
5. $\boxed{T(n) = \Theta(n^2)}$

> **Correção de digitação do slide:** no exemplo do Caso 3 da aula aparece "$T(n) = \Theta(n)$". O correto é $T(n) = \Theta(f(n)) = \Theta(n^2)$.

Mais aplicações rápidas:

Recorrência	$a, b, f(n)$	$n^{\log_b a}$	Caso	Resultado
$T(n/2) + c$	$1, 2, c$	$n^0 = 1$	2	$\Theta(\log n)$
$T(n/2) + n$	$1, 2, n$	1	3 (regularidade: $n/2 \leq \frac{1}{2}n$ ✓)	$\Theta(n)$
$2T(n/2) + 1$	$2, 2, 1$	n	1	$\Theta(n)$
$2T(n/2) + n$	$2, 2, n$	n	2	$\Theta(n \log n)$
$4T(n/2) + n$	$4, 2, n$	n^2	1	$\Theta(n^2)$
$2T(n/2) + n^2$	$2, 2, n^2$	n	3	$\Theta(n^2)$

Quando o Método Mestre **não** se aplica

Recorrência	Por quê	Use
$T(n) = T(n-1) + n$	subtrai em vez de dividir: $T(n-1)$ e não $T(n/b)$	Iteração
$T(n) = T(n/2) + T(n/4) + n$	subproblemas de tamanhos diferentes	Árvore de recursão
$T(n) = T(n-1) + T(n-2) + c$	subtração e tamanhos diferentes	Árvore / limites (seção 8)
$T(n) = 2T(n/2) + n \log n$	cai entre os casos 2 e 3	Árvore (dá $\Theta(n \log^2 n)$)

---

8. Recursão com subproblemas diferentes: Fibonacci (Praticando 7)

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

- **Chamadas por execução:** 2 (para $n > 1$).
- **Tamanhos:** $n-1$ e $n-2$ (diferentes → Mestre não se aplica).
- **Recorrência do tempo:** $T(n) = T(n-1) + T(n-2) + c$.

8.1 Número de chamadas

Contando a chamada atual: $C(n) = C(n-1) + C(n-2) + 1$, com $C(0) = C(1) = 1$.

n	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
$C(n)$	1	1	3	5	9	15	25	41	67	109	177

Somando 1 dos dois lados, $D(n) = C(n) + 1$ satisfaz $D(n) = D(n-1) + D(n-2)$ — a própria sequência de Fibonacci! Fica $C(n) = 2F(n+1) - 1$ (com $F(1) = F(2) = 1$).

A árvore **cresce rapidamente**: muitas chamadas recalculam os mesmos valores (por exemplo, `fib(n-2)` é calculado dentro de `fib(n)` e de novo dentro de `fib(n-1)`).

8.2 Limites passo a passo

Como T é crescente, $T(n-2) \le T(n-1)$.

Limite superior: $T(n) \le 2T(n-1) + c$. Expandindo k vezes:

$$T(n) \le 2^k T(n-k) + c(2^k - 1) \xrightarrow{k=n-1} T(n) \le 2^{n-1}T(1) + c\,2^{n-1} = O(2^n)$$

Limite inferior: $T(n) \ge 2T(n-2) + c \ge 2^k T(n-2k)$. Com $k = n/2$:

$$T(n) \ge 2^{n/2}\,T(0) = \Omega\big((\sqrt{2})^n\big) \approx \Omega(1{,}414^n)$$

Valor exato da base: a parte homogênea $T(n) = T(n-1) + T(n-2)$ tem equação característica $x^2 = x + 1$, cujas raízes são $x = \dfrac{1 \pm \sqrt{5}}{2}$. A maior raiz domina:

$$\boxed{T(n) = \Theta(\varphi^n)}, \qquad \varphi = \frac{1+\sqrt{5}}{2} \approx 1{,}618$$

O limite $O(2^n)$ também é correto, apenas menos preciso.

> **Curiosidade:** guardando os valores já calculados (memoização) ou fazendo um laço, Fibonacci cai para $\Theta(n)$.

---

9. Complexidade de espaço: a pilha de recursão (Praticando 8 e 9)

Cada chamada ativa ocupa um **quadro (frame)** na pilha. O espaço usado é proporcional à **profundidade máxima** da recursão (o maior número de chamadas ativas ao mesmo tempo), e **não** ao número total de chamadas.

```
def conta(n):                # Praticando 8
    if n <= 1:
        return
    conta(n - 1)

def conta(n):                # Praticando 9
    if n <= 1:
        return
    conta(n // 2)
```

Característica	Praticando 8: `conta(n-1)`	Praticando 9: `conta(n//2)`
Redução do problema	$n - 1$	$n/2$
Recorrência	$T(n) = T(n-1) + c$	$T(n) = T(n/2) + c$
Tempo	$\Theta(n)$	$\Theta(\log n)$
Chamadas simultâneas na pilha	$\approx n$: `conta(n)`, `conta(n-1)`, ...	$\approx \log_2 n$: `conta(n)`, `conta(n/2)`, ...
Espaço na pilha	$S(n) = \Theta(n)$	$S(n) = \Theta(\log n)$

Conclusão: reduzir o problema pela metade diminui drasticamente o número de chamadas, tanto em tempo quanto em espaço.

Outros exemplos:

- **Fibonacci:** tempo $\Theta(\varphi^n)$, mas espaço só $\Theta(n)$ (a pilha desce no máximo n níveis por vez).
- **Merge Sort / Quick Sort equilibrado:** pilha $\Theta(\log n)$; **Quick Sort no pior caso:** pilha $\Theta(n)$.

---

10. Qual método usar?

Recorrência	Método
$T(n) = T(n-1) + c$	Iteração
$T(n) = T(n-1) + n$	Iteração
$T(n) = T(n/2) + c$	Iteração (ou Mestre)
$T(n) = 2T(n/2) + n$	Árvore / Mestre
$T(n) = 4T(n/2) + n$	Mestre
Subproblemas de tamanhos diferentes	Árvore de recursão
Necessidade de **demonstrar** a hipótese	Substituição

---

11. Exercícios resolvidos (Praticando 1 a 11)

Praticando 1 — quatro funções

```
def A(n):
    if n <= 1:
        return
    A(n-1)
```

```
def B(n):
    if n <= 1:
        return
    B(n//2)
```

```
def C(n):
    if n <= 1:
        return
    C(n//2)
    C(n//2)
```

```
def D(n):
    if n <= 1:
        return
    D(n//2)
    D(n//2)
    for i in range(n):
        print(i)
```

Função	Chamadas	Tamanho	Trabalho local	Recorrência	Método	Complexidade
A	1	$n-1$	c	$T(n) = T(n-1) + c$	Iteração (seção 4.1)	$\Theta(n)$
B	1	$n/2$	c	$T(n) = T(n/2) + c$	Iteração: $k = \log n$ (seção 4.3)	$\Theta(\log n)$
C	2	$n/2$	c	$T(n) = 2T(n/2) + c$	Mestre: $n^{\log_2 2} = n > c$ → Caso 1	$\Theta(n)$
D	2	$n/2$	n	$T(n) = 2T(n/2) + n$	Mestre: $f(n) = n = n^{\log_2 2}$ → Caso 2	$\Theta(n \log n)$

Conclusão: C e D têm as mesmas chamadas; o que muda a complexidade é o trabalho local.

Praticando 2 — `misterio(n)`: laço de n e depois `misterio(n//2)`

a) **1** chamada recursiva por execução. b) Subproblema de tamanho $n/2$. c) Trabalho local: o laço executa n vezes → $\Theta(n)$. d) $T(n) = T(n/2) + n$, $T(1) = c$. e) Iteração:

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + \cdots + 2 + T(1) \leq 2n + c$$

f) $\boxed{T(n) = \Theta(n)}$ (detalhes na seção 4.4). Mostra que haver recursão não implica $n \log n$.

Praticando 3 — `processo(n)`: chama `processo(n//2)` e `processo(n//4)` + laço de n

a) **2** chamadas recursivas. b) **Não** têm o mesmo tamanho: $n/2$ e $n/4$. c) **Não** diretamente: a forma $aT(n/b) + f(n)$ pressupõe subproblemas iguais. d) $T(n) = T(n/2) + T(n/4) + cn$, custo constante no caso base. e) **Árvore de recursão** (ou expansão), porque os subproblemas são diferentes. Cada nível custa $(3/4)$ do anterior; soma geométrica $\leq 4cn$ → $\boxed{\Theta(n)}$ (seção 5.3).

Praticando 4 — $T(n) = 2T(n/2) + 1$

a) Quatro níveis: 1 nó (n), 2 nós ($n/2$), 4 nós ($n/4$), 8 nós ($n/8$), cada um com custo 1. b) Nível 0: **1** nó. c) Nível 1: **2** nós. d) Nível 2: **4** nós (o número dobra a cada nível: 2^i). e) Custo do nível i: $C_i = 2^i \cdot 1 = 2^i$. f) Termina quando $n/2^k = 1 \Rightarrow k = \log_2 n$ divisões ($\log_2 n + 1$ níveis contando o nível 0). g) $T(n) = 1 + 2 + 4 + \cdots + n = 2n - 1 = \boxed{\Theta(n)}$ (seção 5.2).

Praticando 5 — Algoritmo A: $2T(n/2) + n$ × Algoritmo B: $2T(n/2) + 1$

a) **B** parece mais eficiente: faz trabalho constante fora da recursão; A faz trabalho linear. b) A: Mestre Caso 2 → $\Theta(n \log n)$. B: Mestre Caso 1 → $\Theta(n)$. c) Ambos fazem 2 chamadas de tamanho $n/2$, mas o trabalho local difere: em A cada **nível** da árvore custa n ($\log n$ níveis → $n \log n$); em B o custo é dominado pela **quantidade de nós** da árvore ($\approx 2n$). d) $f(n)$ representa o trabalho realizado na chamada atual, fora das chamadas recursivas (A: $f(n) = n$; B: $f(n) = 1$).

Praticando 6 — uma chamada `exemplo(n//2)` + dois laços de n

a) **1** chamada recursiva. b) Primeiro `for`: n. c) Segundo `for`: n. d) Trabalho local total: $f(n) = n + n = 2n$. e) $T(n) = T(n/2) + 2n$. f) Mestre: $a = 1$, $b = 2$, $f(n) = 2n$; $n^{\log_2 1} = n^0 = 1$. O trabalho local é polinomialmente maior ($2n = \Omega(n^{0+1})$) e vale a regularidade: $1 \cdot f(n/2) = n \leq \frac{1}{2} \cdot 2n$ ✓. Caso 3:

$$\boxed{T(n) = \Theta(n)}$$

Praticando 7 — Fibonacci recursivo

a) Para $n > 1$, **2** chamadas. b) Tamanhos $n - 1$ e $n - 2$. c) $C(n) = C(n-1) + C(n-2) + 1$, com $C(0) = C(1) = 1$. d) Não tem a forma $aT(n/b) + f(n)$: os subproblemas são diferentes e obtidos por subtração. e) **Sim**, cresce exponencialmente: há muitas chamadas repetidas. $T(n) = \Theta(\varphi^n)$, $\varphi \approx 1{,}618$ (ou o limite mais simples $O(2^n)$). Detalhes na seção 8.

Praticando 8 — `conta(n-1)`

a) $T(n) = T(n-1) + c$, $T(1) = c_0$. b) $\approx n$ chamadas → $\Theta(n)$. c) No pior momento, antes de começar a retornar, há $\approx n$ chamadas ativas. d) Cada chamada usa espaço constante: $S(n) = \Theta(n)$.

Praticando 9 — `conta(n//2)`

a) $T(n) = T(n/2) + c$. b) Tamanhos $n, n/2, n/4, \ldots, 1$ → $\approx \log_2 n$ chamadas → $\Theta(\log n)$. c) $\approx \log_2 n$ chamadas simultâneas. d) $S(n) = \Theta(\log n)$. Comparação na tabela da seção 9.

Praticando 10 — "duas chamadas de metade, então é $O(n)$"?

Não concordo. O aluno olhou só o tamanho dos subproblemas e esqueceu o trabalho acumulado em todos os níveis. Mestre: $a = 2$, $b = 2$, $f(n) = n$, $n^{\log_2 2} = n$ → Caso 2 → $T(n) = \Theta(n \log n)$. Pela árvore: cada nível custa n e há $\log n$ níveis. (A tentativa de provar $O(n)$ por substituição falha — seção 6.3.)

Praticando 11 — qual é melhor?

Recorrência	Solução	Complexidade
$A(n) = T(n-1) + 1$	Iteração	$\Theta(n)$
$B(n) = T(n/2) + n$	Mestre Caso 3 / série geométrica	$\Theta(n)$
$C(n) = 2T(n/2) + n$	Mestre Caso 2	$\Theta(n \log n)$

Ordem: $A \sim B < C$. **A e B têm a mesma ordem assintótica**, mas estruturas recursivas completamente diferentes.

Referência de ordens de crescimento:

Ordem	Complexidade	Comportamento típico
1	$\Theta(1)$	custo constante
2	$\Theta(\log n)$	redução pela metade
3	$\Theta(n)$	percorrer os dados uma vez
4	$\Theta(n \log n)$	divisão recursiva com trabalho linear por nível
5	$\Theta(n^2)$	dois laços lineares aninhados
6	$\Theta(2^n)$	recursão exponencial

---

12. Matérias correlatas

Merge Sort — é o Exemplo 3 em forma de algoritmo: $T(n) = 2T(n/2) + \Theta(n) \to \Theta(n \log n)$ por árvore, iteração, substituição e Mestre. Veja 06 — Merge Sort.

Quick Sort — partição equilibrada dá $T(n) = 2T(n/2) + cn$ (Caso 2 → $\Theta(n \log n)$); partição extrema dá $T(n) = T(n-1) + cn$ (iteração → $\Theta(n^2)$, igual ao Exemplo 2). Pilha: $\Theta(\log n)$ × $\Theta(n)$. Veja 07 — Quick Sort.

Heap Sort — o `heapify` recursivo desce no máximo $\log n$ níveis ($T(h) = T(h-1) + c$); a construção do heap usa a série $\sum h/2^h$ do apêndice. Veja 08 — Heap Sort.

Algoritmos quadráticos — Bubble, Selection e Insertion Sort usam a soma de Gauss $1 + 2 + \cdots + (n-1) = n(n-1)/2$ da seção 4.2. Veja 03 — Bubble Sort, 04 — Selection Sort e 05 — Insertion Sort.

Visão geral de ordenação — 02 — Fundamentos e Comparação.

---

13. Apêndice: ferramentas matemáticas

Logaritmos

- $\log_b n = k \iff b^k = n$. Ex.: $\log_2 8 = 3$; $\log_2 1024 = 10$.
- $\log(xy) = \log x + \log y$; $\log(x/y) = \log x - \log y \to \log_2(n/2) = \log_2 n - 1$.
- $\log(x^k) = k \log x$; $a^{\log_a n} = n \to 2^{\log_2 n} = n$.
- Mudança de base: $\log_b n = \dfrac{\log_2 n}{\log_2 b}$ — por isso a base do log não importa dentro de Θ.
- "Quantas vezes divido n por 2 até chegar a 1?" → $\log_2 n$ vezes.

Soma aritmética (Gauss)

$$1 + 2 + \cdots + n = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

Truque: escreva a soma na ordem normal e na inversa; cada par de colunas soma $n+1$, e há n colunas: $2S = n(n+1)$.

Série geométrica

$$\sum_{i=0}^{k} r^i = \frac{r^{k+1} - 1}{r - 1} \quad (r \neq 1)$$

- Se $0 < r < 1$: $\displaystyle\sum_{i=0}^{\infty} r^i = \frac{1}{1-r}$ (constante!) → a soma é dominada pelo **primeiro** termo. Ex.: $r = 1/2 \Rightarrow 2$; $r = 3/4 \Rightarrow 4$.
- Se $r > 1$: a soma é dominada pelo **último** termo. Ex.: $1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1$.

Série $\sum h/2^h$ (usada no Heap Sort)

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1 - 1/2)^2} = 2$$

(Derivando $\sum x^h = \frac{1}{1-x}$ obtém-se $\sum h x^h = \frac{x}{(1-x)^2}$; com $x = 1/2$ dá 2.)

Fatorial e $\log(n!)$ (usado no Heap Sort)

$$\log(n!) = \sum_{i=1}^{n} \log i = \Theta(n \log n)$$

pois $\log(n!) \leq n \log n$ e $\log(n!) \geq \frac{n}{2} \log \frac{n}{2}$ (metade dos termos é pelo menos $\log(n/2)$).

Referências da aula: SERPA et al., *Análise de Algoritmos* (SAGAH, 2021); CORMEN et al., *Algoritmos* (4. ed., GEN LTC, 2024); RODRIGUES et al., *Teoria dos grafos e análise de algoritmos* (SAGAH, 2022).